

1. Rebond d'une balle

Première partie

L'objectif de cet exercice est de résoudre le problème suivant :

Lorsqu'une balle tombe d'une hauteur initiale h , sa vitesse à l'arrivée au sol est $v = \sqrt{2 * h * g}$.

Immédiatement après le rebond, sa vitesse est $v1 = \text{eps} * v$ (où **eps** est une constante et v la vitesse avant le rebond). Elle remonte alors à la hauteur $h = \frac{v1^2}{2g}$.

Le but est d'écrire un programme (Rebonds1.java) qui calcule la hauteur à laquelle la balle remonte après un nombre **nbr** de rebonds.

Méthode :

On veut résoudre ce problème, non pas du point de vue formel (équations) mais par **simulation** du système physique (la balle).

Utilisez une itération **for** et des variables **v**, **v1**, (les vitesses avant et après le rebond), et **h**, **h1** (les hauteurs au début de la chute et à la fin de la remontée).

Tâches :

Écrivez le programme Rebonds1.java qui affiche la hauteur après le nombre de rebonds spécifié.

Votre programme devra utiliser la **constante g**, de valeur 9,81 et demander à l'utilisateur d'entrer les valeurs de :

- **H0** (hauteur initiale, contrainte : $H0 \geq 0$),
- **eps** (coefficient de rebond, contrainte $0 \leq \text{eps} < 1$)
- **nbr** (nombre de rebonds, contrainte : $0 \leq \text{NBR}$).

Essayez les valeurs $H0 = 25$, $\text{eps} = 0.9$, $\text{NBR} = 10$. **La hauteur obtenue devrait être environ 3.04.**

Remarque :

- Pour utiliser les fonctions mathématiques (comme **Math.sqrt()**), ajoutez **import java.lang.Math** au début de votre fichier source.
- Pour déclarer la constante **g**, utilisez la tournure suivante:

```
public static final double GRAVITE = 9.81;
```

(à placer en dessus de la méthode **main**).

Ceci permet de déclarer une *constante* de portée globale. Nous reviendrons sur les modificateurs **static** et **final** dans notre futur cours sur l'orienté objet.

Deuxième partie

On se demande maintenant combien de rebonds fait cette balle avant que la hauteur à laquelle elle rebondit soit plus petite que (ou égale à) une hauteur donnée h_fin .

Écrivez le programme Rebonds2.java qui affiche le nombre de rebonds à l'écran.

Il devra utiliser une boucle **do...while**, et demander à l'utilisateur d'entrer les valeurs de :

- **H0** (hauteur initiale, contrainte : $H0 \geq 0$),
 - **eps** (coefficient de rebond, contrainte $0 \leq \text{eps} < 1$)
-

- **h_fin** (hauteur finale désirée, contrainte : $0 < h_{fin} < H_0$).

Essayez les valeurs **H0=10**, **eps=0.9** et **h_fin=2**. Vous devriez obtenir **8 rebonds**.

2. Crible d'Ératosthène

Un nombre est dit premier s'il admet exactement 2 diviseurs *distincts* (1 et lui-même). 1 n'est donc pas premier.

Le crible d'Ératosthène est une méthode de recherche des nombres premiers plus petits qu'un entier naturel n donné. Cette méthode est simple:

- On commence par supprimer tous les multiples de 2 inférieurs à n .
- L'entier 3 n'a pas été supprimé et il ne peut être multiple des entiers qui le précèdent, sinon on l'aurait supprimé; il est donc premier. Supprimons alors tous les multiples de 3 inférieurs à n .
- L'entier 5 n'a pas été supprimé, il est donc premier. Supprimons tous les multiples de 5 inférieurs à n .
- Et ainsi de suite jusqu'à n . Les valeurs n'ayant pas été supprimées sont les nombres entiers plus petits que n .

Écrivez le code qui applique cette méthode pour trouver les nombres premiers inférieurs à 100. Vous devez trouver: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97.

On utilisera un tableau de booléens:

```
boolean[] supprime = new boolean[100];
```

pour mémoriser les entiers qui ont été supprimés. N'oubliez pas d'initialiser chacun de ses éléments à `false`.

3. Plus grand diviseurs commun

Ecrivez un programme `PGDC.java` qui calcule et affiche le plus grand diviseur commun de deux nombres entiers positifs entrés au clavier. Exemples d'exécution du programme:

```
Entrez un nombre positif : 9
Entrez un nombre positif : 6
Le plus grand diviseur commun de 9 et 6 est 3

Entrez un nombre positif : 9
Entrez un nombre positif : 4
Le plus grand diviseur commun de 9 et 4 est 1
```

Utilisez l'algorithme d'Euclide pour déterminer le plus grand diviseur. Cette formule se résume comme suit:

Soient deux nombres entiers positifs a et b . Si a est plus grand que b , le plus grand diviseur commun de a et b est le même que pour $a-b$ et b . Vice versa si b est plus grand que a .

Les équivalences mathématiques utiles sont:

1. Si $a > b$, alors $\text{PGDC}(a, b) = \text{PGDC}(a-b, b)$
2. $\text{PGDC}(a, a) = a$

Exemple de calcul de $\text{PGDC}(42, 24)$:

1. $42 > 24$, alors $\text{PGDC}(42, 24) = \text{PGDC}(42-24, 24) = \text{PGDC}(18, 24) = \text{PGDC}(24, 18)$
2. $24 > 18$, alors $\text{PGDC}(24, 18) = \text{PGDC}(24-18, 18) = \text{PGDC}(6, 18) = \text{PGDC}(18, 6)$
3. $18 > 6$, alors $\text{PGDC}(18, 6) = \text{PGDC}(18-6, 6) = \text{PGDC}(12, 6)$
4. $12 > 6$, alors $\text{PGDC}(12, 6) = \text{PGDC}(12-6, 6) = \text{PGDC}(6, 6)$
5. Résultat: $\text{PGDC}(42, 24) = \text{PGDC}(6, 6) = 6$

Indication: utilisez une boucle (par exemple `while`) qui s'occupe de modifier et de tester les valeurs de a et b jusqu'à ce qu'une solution soit trouvée
